3. If it accepts, *reject*. If it rejects, *accept*."

D is a decider because each M_i is a decider. But D doesn't appear on the enumeration for A, because it differs from M_i on input s_i.

4.22 Let K be the enumerator for B. The following TM L is the lexicographic-order enumerator for C.

L = "Ignore the input.

1. Enumerate machines M_i by K.
2. For each M_i, insert useless states, symbols or transitions to obtain N_i such that $L(M_i) = L(N_i)$ and $\langle N_i \rangle$ is lexicographically larger than all $\langle N_j \rangle$ for $1 \leq j < i$. Output N_i."

Let $\langle N_1 \rangle, \langle N_2 \rangle \ldots$ be the strings enumerated by L. Since $L(N_i) = L(M_i)$ for all i, every machine described in B has an equivalent one in C and vice versa. C is decidable because it enumerated by L in lexicographic order, by virtue of Problem 3.16.

Chapter 5

5.1 Suppose for a contradiction that EQ_{CFG} were decidable. We construct a decider M for $ALL_{\mathsf{CFG}} = \{\langle G \rangle | \ G \text{ is a CFG and } L(G) = \Sigma^*\}$ as follows:

M = "On input $\langle G \rangle$:

1. Construct a CFG H such that $L(H) = \Sigma^*$.
2. Run the decider for EQ_{CFG} on $\langle G, H \rangle$.
3. If it accepts, *accept*. If it rejects, *reject*."

M decides ALL_{CFG} assuming a decider for EQ_{CFG} exists. Since we know ALL_{CFG} is undecidable, we have a contradiction.

5.2 Here is a Turing Machine M which recognizes the complement of EQ_{CFG}:

M = "On input $\langle G, H \rangle$:

1. Lexicographically generate the strings $x \in \Sigma^*$.
2. For each such string x:
3. Test whether $x \in L(G)$ and whether $x \in L(H)$, using the algorithm for A_{CFG}.
4. If one of the tests accepts and the other rejects, *accept*; otherwise, continue."

5.3 Here is a match: $\left[\frac{\mathtt{ab}}{\mathtt{abab}}\right]\left[\frac{\mathtt{ab}}{\mathtt{abab}}\right]\left[\frac{\mathtt{aba}}{\mathtt{b}}\right]\left[\frac{\mathtt{b}}{\mathtt{a}}\right]\left[\frac{\mathtt{b}}{\mathtt{a}}\right]\left[\frac{\mathtt{aa}}{\mathtt{a}}\right]\left[\frac{\mathtt{aa}}{\mathtt{a}}\right]$.

5.4 No, it does not imply that A is regular. For example, $\{\mathtt{a}^n\mathtt{b}^n\mathtt{c}^n \mid n \geq 0\} \leq_\mathrm{m} \{\mathtt{a}^n\mathtt{b}^n \mid n \geq 0\}$. The reduction first tests whether its input is a member of $\{\mathtt{a}^n\mathtt{b}^n\mathtt{c}^n \mid n \geq 0\}$. If so, it outputs the string $\mathtt{ab}$, and if not, it outputs the string $\mathtt{a}$.

5.5 Suppose for a contradiction that $A_{\mathsf{TM}} \leq_\mathrm{m} E_{\mathsf{TM}}$ via reduction f. It follows from the definition of mapping reducibility that $\overline{A_{\mathsf{TM}}} \leq_\mathrm{m} \overline{E_{\mathsf{TM}}}$ via the same reduction function f. Observe that $\overline{E_{\mathsf{TM}}}$ is Turing-recognizable, but, $\overline{A_{\mathsf{TM}}}$ is not Turing-recognizable, contradicting Theorem 5.22.

5.6 Suppose $A \leq_\mathrm{m} B$ and $B \leq_\mathrm{m} C$. Then there are computable functions f and g such that $x \in A \iff f(x) \in B$ and $y \in B \iff g(y) \in C$. Consider the function composition $h(x) = g(f(x))$. We can build a TM which computes h as follows: first simulate a TM for f (such a TM exists since we assumed that f is computable) on input x and call the output y. Then simulate a TM for g on y. The output is $h(x) = g(f(x))$. Therefore, h is a computable function. Moreover, $x \in A \iff h(x) \in C$. Therefore, $A \leq_\mathrm{m} C$ via the reduction function h.

5.7 Suppose $A \leq_\mathrm{m} \overline{A}$. Then, $\overline{A} \leq_\mathrm{m} A$, by the definition of mapping reducibility. Because A is Turing-recognizable, Theorem 5.22 implies that $\overline{A}$ is Turing-recognizable, and then Theorem 4.16 implies that A is decidable.

5.8 We need to handle the case where the head is at the leftmost tape call and attempts to move left. To do so we add dominos

$$\left[\frac{\#qa}{\#rb}\right]$$

For every $q, r \in Q$ and $a, b \in \Gamma$, where $\delta(q, a) = (r, b, \mathrm{L})$.

5.9 Suppose L is a Turing-recognizable language and let M be a TM that recognizes it. To reduce L to A_{TM} we map any string x to $\langle M, x\rangle$. Then $x \in L \iff \langle M, x\rangle \in A_{\mathsf{TM}}$. In addition, this mapping is computable, so it gives a mapping reduction from L to A_{TM}.

5.10 To prove that J is not Turing-recognizable, we show that $\overline{A_{\mathsf{TM}}} \leq_\mathrm{m} J$. The reduction function maps any string y to the string $1y$. Then $y \in \overline{A_{\mathsf{TM}}} \iff 1y \in J$. This mapping is computable, so we have shown that $\overline{A_{\mathsf{TM}}} \leq_\mathrm{m} J$. To show that $\overline{J}$ is not Turing-recognizable we show that $A_{\mathsf{TM}} \leq_\mathrm{m} J$ (since $A_{\mathsf{TM}} \leq_\mathrm{m} J \iff \overline{A_{\mathsf{TM}}} \leq_\mathrm{m} \overline{J}$). The reduction function maps any string x to $0x$. Then $x \in A_{\mathsf{TM}} \iff 0x \in J$. This mapping is computable, so we have shown that $A_{\mathsf{TM}} \leq_\mathrm{m} J$.

5.11 Use the language J from Problem 5.10. That problem showed that J is undecidable. Here we show that $J \leq_m \overline{J}$. The reduction f maps ε to itself and for other strings x,

$$f(x) = \begin{cases} 0s & \text{if } x = 1s \text{ for some string } s \in \Sigma^* \\ 1s & \text{if } x = 0s \text{ for some string } s \in \Sigma^* \end{cases}$$

The function f is computable, and $x \in J \iff f(x) \in \overline{J}$. Thus, $J \leq_m \overline{J}$.

5.12 We show that $A_{\mathsf{TM}} \leq_m S$ by mapping $\langle M, w \rangle$ to $\langle M' \rangle$ where M' is the following TM:

M' = "On input x:

1. If $x = 01$ then *accept*.
2. If $x \neq 10$ then *reject*.
3. If $x = 10$ simulate M on w. If M accepts w then *accept*; if M halts and rejects w then *reject*."

If $\langle M, w \rangle \in A_{\mathsf{TM}}$ then M accepts w and $L(M') = \{01, 10\}$, so $\langle M' \rangle \in S$. Conversely, if $\langle M, w \rangle \notin A_{\mathsf{TM}}$ then $L(M') = \{01\}$, so $\langle M' \rangle \notin S$. Therefore, $\langle M, w \rangle \in A_{\mathsf{TM}} \iff \langle M' \rangle \in S$.

5.13 Let $USELESS_{\mathsf{TM}} = \{\langle M \rangle \mid M \text{ is a TM with 1 or more useless states}\}$. We show $USELESS_{\mathsf{TM}}$ is undecidable by reducing E_{TM} to it. Let R be a TM that decides $USELESS_{\mathsf{TM}}$ and construct a TM S that decides E_{TM} as follows.

S = "On input $\langle M, w \rangle$:

1. First we construct a TM N that simulates M on w but ensuring that all of M's original states are not useless. The TM N has input alphabet $\{0, 1, 2\}$. On input 0, N goes through each of the original states of M, except for q_{accept} and q_{reject}, until all nonhalting states of M have been used, and then enters state q_{accept}. On input 1, N enters state q_{reject}. On input 2, N simulates M on w. Finally N restores the original symbol to where the new symbol had been written, and branches to M's start state to simulate M. If that simulation is about to enter M's accept state, it instead enters a new state r.
2. Run $USELESS_{\mathsf{TM}}$ on input $\langle N \rangle$. If it accepts, *reject*. If it rejects, *accept*."

The only possible useless state of N is the state r, because on inputs 0 and 1 it uses all other states. The only way for N to use state r, is if M accepts w. So, testing whether N has any useless states, in effect tests whether M accepts w. Note that the hardest part of this problem is making sure that the machine N has no "unexpected" useless states.

5.14 $L_{\mathsf{TM}} = \{\langle M, w\rangle |\ M$ on w tries moving its head left when on the leftmost tape cell$\}$. This language is undecidable. To prove it, we show that if it was decidable by some machine say R then we could decide A_{TM} with the following machine S:

$S =$ "On input $\langle M, w\rangle$:

1. Convert M to M', where M' first moves its input over one square to the right and puts a special delimiter on the leftmost tape cell. Then M' simulates M on the input. If M' ever sees a delimiter (M must have tried to move left when on the leftmost tape cell), M' automatically moves right and stays in the same state. If M accepts, it moves its head all the way to the left and then moves left off the leftmost tape cell.
2. Run R, the decider for L_{TM}, on $\langle M', w\rangle$.
3. If R accepts then *accept*. If it rejects, *reject*."

TM S decides A_{TM} since the only way M' moves left from the leftmost tape cell is when M accepts w. S never attempts the move during the course of the simulation because we put the special delimiter to "intercept" such moves if made by M.

5.15 Let $LM_{\mathsf{TM}} = \{\langle M, w\rangle |\ M$ ever moves left while computing on $w\}$. LM_{TM} is decidable. Let $M_{\mathbf{LEFT}}$ be the TM which on input $\langle M, w\rangle$ determines the number of states $n_M = |Q_M|$ of M and then simulates M on w for $|w|+n_M+1$ steps. If $M_{\mathbf{LEFT}}$ discovers that M moves left during that simulation, $M_{\mathbf{LEFT}}$ accepts $\langle M, w\rangle$. Otherwise $M_{\mathbf{LEFT}}$ rejects $\langle M, w\rangle$.

The reason that $M_{\mathbf{LEFT}}$ can reject without simulating M further is as follows. Suppose M does make a left move on input w. Let $p = q_0, q_1, \ldots, q_s$ be the shortest computation path of M on w ending in a left move. Because M has been scanning only blanks (it's been moving right) since state $q_{|w|}$, we may remove any cycles that appear after this state and be left with a legal computation path of the machine ending in a left move. Hence p has no cycles and must have length at most $|w|+n_M+1$. Hence $M_{\mathbf{LEFT}}$ would have accepted $\langle M, w\rangle$, as desired.

5.16 Let $B = \{\langle M\rangle |\ M$ is a two tape TM which writes a non-blank symbol on its second tape when it is simulated on some particular input $w\}$. We show that $A_{\mathsf{TM}} \leq_{\mathrm{m}} B$ by mapping $\langle M, w\rangle$ to $\langle M'\rangle$ where M' has the following description:

$M' =$ "On input x:

1. Simulate M on w only using the first tape.
2. If M accepts w then write `blah blah !` on the second tape."

Observe that $\langle M, w\rangle \in A_{\mathsf{TM}} \iff M' \in B$.

5.17 The *PCP* over a unary alphabet is decidable. We describe a TM M that decides unary *PCP*. Given a unary *PCP* instance

$$\left\{\left[\frac{1^{a_1}}{1^{b_1}}\right], \ldots, \left[\frac{1^{a_n}}{1^{b_n}}\right]\right\}$$

M = "On input $\langle a_1, b_1, \ldots, a_n, b_n\rangle$:

1. Check if $a_i = b_i$ for some i. If so, *accept*.
2. Check if there exist i, j such that $a_i > b_i$ and $a_j < b_j$. If so, *accept*. Otherwise, *reject*."

In the first stage, M first checks for a single domino which forms a solution. In the second stage, M looks for two dominos which form a solution. If it finds such a pair, it can construct a solution by picking $(b_j - a_j)$ pieces of the i^{th} domino, putting them together with $(a_i - b_i)$ pieces of the j^{th} domino. This construction has $a_i(b_j - a_j) + a_j(a_i - b_i) = a_i b_j - a_j b_i$ 1s on the top, and $b_i(b_j - a_j) + b_j(a_i - b_i) = a_i b_j - a_j b_i$ 1s on the bottom. If neither stages of M accept, the problem instance contains dominos with all the upper parts having more (or less) 1s than the lower parts. In such a case no solution can exist. Therefore M rejects.

5.18 We construct a computable function that takes a *PCP* instance of any finite alphabet and produces a binary alphabet *PCP* (*BPCP*) instance. As a result we show that *BPCP* is undecidable.

For any *PCP* instance A over a finite alphabet $\{a_1, \ldots, a_n\}$, the function encodes each character a_i into binary as 10^i. The function is a mapping reduction from the general *PCP* problem to the *BPCP* problem. For any P in *PCP*, the converted instance P' is in *BPCP* because a match for P becomes a match for P' using the same set of dominos after conversion. Conversely, if P' is in *BPCP*, P is in *PCP*, because we can uniquely decode a match in P' to a match in P. Therefore, the function is a mapping reduction from *PCP* to *BPCP*.

5.19 We reduce *PCP* to $AMBIG_{\mathsf{CFG}}$, thereby proving that $AMBIG_{\mathsf{CFG}}$ is undecidable. We use the construction given in the hint in the text.

Note: The hint that appears in the first printing is incorrect. The correct version (which appears in the second and subsequent printings) is: Given an instance

$$P = \left\{\left[\frac{t_1}{b_1}\right], \left[\frac{t_2}{b_2}\right], \cdots, \left[\frac{t_k}{b_k}\right]\right\},$$

$$\begin{aligned} S &\to T \mid B \\ T &\to t_1 T \mathbf{a}_1 \mid \cdots \mid t_k T \mathbf{a}_k \mid t_1 \mathbf{a}_1 \mid \cdots \mid t_k \mathbf{a}_k \\ B &\to b_1 B \mathbf{a}_1 \mid \cdots \mid b_k B \mathbf{a}_k \mid b_1 \mathbf{a}_1 \mid \cdots \mid b_k \mathbf{a}_k \ , \end{aligned}$$

where $\mathbf{a}_1, \ldots, \mathbf{a}_k$ are new terminal symbols. We prove that this reduction works.

We show that P has a solution iff the CFG is ambiguous. If P has a match $t_{i_1}t_{i_2}\ldots t_{i_l} = b_{i_1}b_{i_2}\ldots b_{i_l}$, the string

$$t_{i_1}t_{i_2}\ldots t_{i_l}\mathbf{a}_{i_l}\ldots\mathbf{a}_{i_2}\mathbf{a}_{i_1} = b_{i_1}b_{i_2}\ldots b_{i_l}\mathbf{a}_{i_l}\ldots\mathbf{a}_{i_2}\mathbf{a}_{i_1}$$

has two different leftmost derivations, one from T and one from B. Hence the CFG is ambiguous.

Conversely, if the CFG is ambiguous, some string w has multiple leftmost derivations. All generated strings have the form $w = w_{start}\mathbf{a}_{i_l}\ldots\mathbf{a}_{i_2}\mathbf{a}_{i_1}$ where w_{start} contains only symbols from P's alphabet. Notice that once we choose the first step in the derivation of w (either $S \to T$ or $S \to B$), the following steps in the derivation are uniquely determined by the sequence $\mathbf{a}_{i_l}\ldots\mathbf{a}_{i_2}\mathbf{a}_{i_1}$. Therefore w has at most two leftmost derivations:

i) $S \to T \to t_{i_1}T\mathbf{a}_{i_1} \to t_{i_1}t_{i_2}T\mathbf{a}_{i_2}\mathbf{a}_{i_1} \to \cdots \to t_{i_1}t_{i_2}\ldots t_{i_l}\mathbf{a}_{i_l}\ldots\mathbf{a}_{i_2}\mathbf{a}_{i_1}$, and

ii) $S \to B \to b_{i_1}T\mathbf{a}_{i_1} \to b_{i_1}b_{i_2}T\mathbf{a}_{i_2}\mathbf{a}_{i_1} \to \cdots \to b_{i_1}b_{i_2}\ldots b_{i_l}\mathbf{a}_{i_l}\ldots\mathbf{a}_{i_2}\mathbf{a}_{i_1}$.

If w is ambiguous, $t_{i_1}t_{i_2}\ldots t_{i_l}\mathbf{a}_{i_l}\ldots\mathbf{a}_{i_2}\mathbf{a}_{i_1} = b_{i_1}b_{i_2}\ldots b_{i_l}\mathbf{a}_{i_l}\ldots\mathbf{a}_{i_2}\mathbf{a}_{i_1}$, and therefore $t_{i_1}t_{i_2}\ldots t_{i_l} = b_{i_1}b_{i_2}\ldots b_{i_l}$. Thus, the initial PCP instance, P, has a match.

5.20

a. A 2DFA M with s states computing on an input x of size n has at most $s(n+2)^2$ possible configurations. Thus, on input x, M either halts in $s(n+2)^2$ steps or loops forever. To decide A_{2DFA}, it is enough to simulate M on x for $s(n+2)^2$ steps and accept if M has halted and accepted x during this finite duration.

b. Suppose a TM D decides E_{2DFA}. We construct a TM E that decides E_{TM}. E runs as follows:

E = "On input $\langle M\rangle$,

1. Construct a 2DFA M' that recognizes the language of accepting computation histories on M, namely, $\{\langle c_1\#c_2\#\ldots\#c_k\rangle |\ c_i$ is a configuration of M, c_1 is a start configuration, c_k is an accepting configuration, c_{i+1} legally follows c_i for each $i\}$. A 2DFA can check whether c_1 is a start configuration of M by verifying that it begins with a start state q_0 and contains legal symbols from M's input alphabet. Similarly, it can check whether c_k is an accepting configuration by verifying that it contains q_{accept} and symbols from M's tape alphabet. These two steps use only one head. To check whether c_{i+1} legally follows c_i, the 2DFA can keep its two heads on c_i and c_{i+1}, and compare them symbol by symbol. Notice that $L(M') = \emptyset$ iff $L(M) = \emptyset$.
2. Run D on $\langle M'\rangle$. If D accepts, *accept*. If D rejects, *reject*."

Since E_{TM} is undecidable, E_{2DFA} is decidable.

5.21 We formulate the problem of testing 2DIM-DFA equivalence as a language: $EQ_{\text{2DIM-DFA}} = \{\langle M_1, M_2\rangle |\ M_1 \text{ is equivalent to } M_2\}$.
We show that if $EQ_{\text{2DIM-DFA}}$ is decidable, we can decide A_{TM} by reducing A_{TM} to $EQ_{\text{2DIM-DFA}}$. Therefore $EQ_{\text{2DIM-DFA}}$ is undecidable. Assume M is the TM deciding $EQ_{\text{2DIM-DFA}}$. We construct S that decides A_{TM} as follows:

S = "On input $\langle A, w\rangle$,

1. Construct a 2DIM-DFA T that accepts only the accepting computation history of A on w. The input that T accepts should represent the computation history of A in the form of an $m \times n$ rectangle, where $(m-2)$ is the length of the longest configuration that A ever has during the computation and $(n-2)$ is the number of steps it takes to get to the accepting state. Each row of the inner cells contains a proper configuration of A during its computation. In order to check if the input is a computation history of A on w, T follows a procedure similar to that given the proof of Theorem 5.9. It first checks that the first inner row contains the configuration with input w and the start state at the left end. Then it checks whether the TM reaches the accept state at the last inner row. After that, it compares two consecutive rows of input to see if the two configurations follow the transition function of A. If the input passes all the three tests, the 2DIM-DFA accepts. Otherwise, it rejects.
2. Construct another 2DIM-DFA E which always rejects. Run M on $\langle T, E\rangle$ to see if T and E are equivalent. If M accepts, *reject* because there is no computation history for A on w. Otherwise, *accept*."

The TM S decides A_{TM}. However, we know that A_{TM} is undecidable. Therefore $EQ_{\text{2DIM-DFA}}$ is also undecidable.

5.22 Suppose P is a decidable language satisfying the properties and let T_P be a TM that decides P. Without loss of generality assume $\langle T_\emptyset\rangle \notin P$ with $L(T_\emptyset) = \emptyset$ since we can consider $\overline{P}$ if $\langle T_\emptyset\rangle$ is in P. Since P is not trivial, there exists a TM M_1 with $\langle M_1\rangle \in P$. We can decide A_{TM} by constructing the following TM:

"On input $\langle M, w\rangle$:

1. Construct a TM M_w that accepts on input x iff M accepts w and M_1 accepts x.
2. Run T_P on $\langle M_w\rangle$. If it accepts, *reject*. Otherwise, *accept*."

If $w \in L(M)$, $L(M_w) = L(M_1)$ and M_w should be accepted by T_P since $\langle M_1\rangle \in P$. If $w \notin L(M)$, $L(M_w) = \emptyset = L(T_\emptyset)$ and should be rejected by the assumption that $\langle T_\emptyset\rangle \notin P$. Therefore we have $w \in L(M)$ iff $\langle M_1\rangle \in P$. However, since A_{TM} is undecidable, P is not decidable either.

5.23 First, let P be the language $\{\langle M\rangle |\ M$ is a TM with 5 states$\}$. P is non-trivial, and so it satisfies the second condition of Rice's Theorem but P can be easily decided by checking the number of states of the input Turing Machine. Second, let P be the empty set. Then it does not contain any TM and so it satisfies the first condition of Rice's Theorem, but and P can be decided by a TM that always rejects.

Therefore both properties are necessary for proving P undecidable.

Chapter 6

6.1 The following program is in LISP:

```
((LAMBDA (X) (LIST X (LIST (QUOTE QUOTE) X)))
(QUOTE (LAMBDA (X) (LIST X (LIST (QUOTE QUOTE) X)))))
```

6.8 We build up two incomparable sets A and B to kill off all possible Turing-reductions, $R_1, R_2, \ldots$. The sets start off undefined.

First, extend A to be longer and longer initial portions of the empty set and B to be longer and longer initial portions of ATM. Eventually, R_1 must get the wrong answer when trying to reduce B to A. Then extend A and B interchanging the empty set and ATM until R_1 errs in reducing A to B. Continue with R_2, etc.

Chapter 7

7.1
a. True
b. False
c. False
d. True
e. True
f. True

7.2
a. False
b. True
c. True
d. True
e. False
f. False

7.3 We can use the Euclidean algorithm to find the greatest common divisor.